

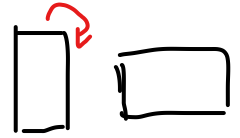
웹: 브라우저 실행, OS 종속X

앱: 브라우저x, 단독실행, OS에 종속

phone
tablet
...

해상도 상이, 종류 많음.

요즘에는 앱 먼저 먼저 개발, 그 다음에 웹 개발(요즘은 모바일 더 많이 씬.)



7장. 반응형 웹 디자인

웹 페이지를 실제로 보여주는 영역

학습 목표 - viewport 메타 태그로 반응형 기반을 설정할 수 있습니다 - 미디어 쿼리와 브레이크포인트로 화면별 스타일을 적용할 수 있습니다 - 모바일 우선 전략으로 레이아웃을 설계할 수 있습니다 - 유연한 단위와 반응형 이미지로 어떤 화면에서도 보기 좋은 페이지를 만들 수 있습니다

질의

반응형 디자인과 뷰포트

도입

오늘날 웹 페이지는 스마트폰, 태블릿, 노트북, 대형 모니터 등 매우 다양한 화면에서 열립니다. 같은 페이지가 어느 기기에서나 읽기 좋게 보이려면 화면 크기에 맞춰 자동으로 레이아웃을 바꾸는 기술이 필요합니다. 이 절에서는 반응형 웹 디자인(Responsive Web Design)이 무엇인지, 그리고 그 출발점인 뷰포트(Viewport) 설정을 배웁니다.

핵심 개념 설명

반응형 디자인 (Responsive Design)

반응형 디자인(Responsive Design)은 하나의 HTML-CSS 코드로 다양한 화면 크기에 맞게 레이아웃이 자동 조정되는 방식입니다. 물이 담긴 그릇 모양에 따라 자연스럽게 형태를 바꾸듯, 콘텐츠가 화면 크기에 맞게 흘러가도록 설계합니다.

반응형 디자인이 등장하기 전에는 모바일용과 데스크톱용 사이트를 따로 만들었습니다. 두 별개의 코드를 유지하는 비용이 크고, 기기 종류가 다양해지면서 더 이상 이 방식이 현실적이지 않습니다.

뷰포트 (Viewport)

뷰포트(Viewport)는 브라우저가 웹 페이지를 실제로 보여 주는 영역입니다. **데스크톱**에서는 브라우저 창 크기가 뷰포트이고, **모바일**에서는 화면 크기가 뷰포트입니다.

viewport 메타 태그

내용
<meta name="viewport" content="width=device-width, initial-scale=1">
장치너비 1배

항목	내용
설명	모바일 브라우저의 뷰포트 크기와 초기 배율을 설정합니다
content 속성	<code>width=device-width</code> : 뷰포트 너비를 기기 화면 너비로 설정 <code>initial-scale=1</code> : 초기 확대/축소 배율을 1(100%)로 고정
위치	<code><head></code> 안에 위치해야 합니다
사용 예시	<code><meta name="viewport" content="width=device-width, initial-scale=1"></code>

스마트폰 화면은 실제 물리 픽셀(Physical Pixel) 밀도가 매우 높습니다. 예를 들어 iPhone의 레티나 디스플레이는 1인치당 픽셀 수가 데스크톱보다 2~3배 많습니다. 그래서 1개의 **CSS 픽셀(CSS Pixel)**이 실제로는 2~3개의 물리 픽셀로 렌더링됩니다. 이 비율을 **장치 픽셀 비율(Device Pixel Ratio, DPR)**이라고 합니다.

코드로 확인하기

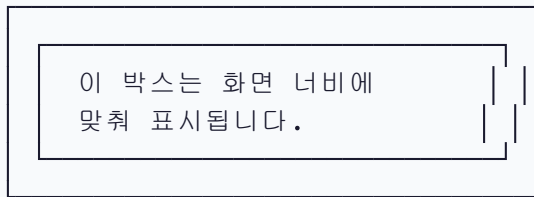
viewport 메타 태그의 유무가 모바일 화면 렌더링에 어떤 차이를 만드는지 확인합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <!-- viewport 메타 태그를 포함해야 모바일에서 올바르게 표시됩니다 -->
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>뷰포트 예제</title>
  <style>
    body {
      font-family: sans-serif;
      margin: 0;
      padding: 16px;
      background-color: #f0f4ff;
    }

    .box {
      background-color: #4f46e5;
      color: white;
      padding: 20px;
      border-radius: 8px;
      text-align: center;
      font-size: 1.2rem;
    }
  </style>
</head>
<body>
  <div class="box">
    이 박스는 화면 너비에 맞춰 표시됩니다.
  </div>
</body>
</html>
```

브라우저 렌더링 결과

[모바일 화면 - viewport 메타 태그 있음]



→ 텍스트가 읽기 편한 크기로 표시됩니다.

[모바일 화면 - viewport 메타 태그 없음]



→ 페이지 전체가 축소되어 글자가 매우 작게 보입니다.

줄별 코드 설명

줄	코드	설명
5	<code><meta name="viewport" content=...</code>	뷰포트 너비를 기기 화면 너비로 설정하고 초기 배율을 100%로 고정합니다
12	<code>margin: 0;</code>	브라우저 기본 여백을 제거하여 전체 너비를 온전히 활용합니다
13	<code>padding: 16px;</code>	좌우 여백을 주어 작은 화면에서도 텍스트가 화면 끝에 붙지 않게 합니다

베스트 프랙티스: `viewport` 메타 태그는 반응형 프로젝트의 모든 HTML 파일 `<head>` 안에 반드시 포함합니다. 이 한 줄이 없으면 이후에 작성하는 미디어 쿼리가 제대로 동작하지 않습니다.

절 요약

- **반응형 디자인(Responsive Design)**은 하나의 코드로 모든 화면 크기를 지원합니다
- **뷰포트(Viewport)**는 브라우저가 페이지를 렌더링하는 실제 영역입니다

- `<meta name="viewport" content="width=device-width, initial-scale=1">` 은 모든 반응형 페이지의 필수 설정입니다

미디어 쿼리와 브레이크포인트

도입

뷰포트 설정만으로는 부족합니다. 모바일에서는 1열로, 태블릿에서는 2열로, 데스크톱에서는 3열로 레이아웃을 바꾸려면 화면 크기 조건에 따라 서로 다른 CSS를 적용하는 방법이 필요합니다. 이것이 바로 미디어 쿼리(Media Query)입니다.

핵심 개념 설명

미디어 쿼리 (Media Query)

미디어 쿼리(Media Query)는 `@media` 규칙을 사용하여 특정 조건(화면 너비, 방향, 해상도 등)을 만족할 때만 CSS를 적용하는 기능입니다. 옷장을 계절에 따라 다르게 정리하듯, 화면 크기에 따라 다른 스타일을 입힙니다.

```
@media (조건) {  
    /* 조건을 만족할 때만 적용되는 CSS */  
}
```

가장 많이 사용하는 조건은 화면 너비(`width`)입니다.

```
        최소 너비  
/* 화면 너비가 768px 이상일 때 */  
@media (min-width: 768px) {  
    .container {  
        display: grid;  
        grid-template-columns: 1fr 2fr;  
    }  
}
```

@media (조건) { ... }

항목	내용
설명	미디어 특성 조건에 따라 CSS 블록을 조건부 적용합니다
주요 조건	<code>min-width: Xpx</code> — 화면 너비가 X픽셀 이상일 때
	<code>max-width: Xpx</code> — 화면 너비가 X픽셀 이하일 때
	<code>screen</code> — 화면(스크린) 미디어 타입에만 적용
AND 조합	<code>@media screen and (min-width: 768px)</code>
사용 예시	<code>@media (min-width: 600px) { .col { flex: 1; } }</code>

브레이크포인트 (Breakpoint)

브레이크포인트(Breakpoint)는 레이아웃이 바뀌는 화면 너비 기준점입니다. 미디어 쿼리를 작성할 때 어떤 너비에서 레이아웃을 변경할지 정하는 것입니다.

팁: 브레이크포인트는 특정 기기 해상도(예: iPhone = 375px)에 맞추는 것이 아니라, **콘텐츠가 보기 불편해지는 지점**을 기준으로 정하는 것이 좋습니다. 기기는 계속 바뀌지만 콘텐츠 흐름은 일관됩니다.

일반적으로 많이 사용하는 브레이크포인트는 다음과 같습니다.

구분	화면 너비	주요 기기
모바일	~ 767px	스마트폰 세로 방향
태블릿	768px ~ 1023px	태블릿, 스마트폰 가로 방향
데스크톱	1024px ~	노트북, 모니터

반응형 브레이크포인트와 화면 구간별 레이아웃 변화

코드로 확인하기

화면 너비에 따라 카드 레이아웃이 1열 → 2열 → 3열로 바뀌는 예제입니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>미디어 쿼리 예제</title>
  <style>
    /* 기본 스타일 (모바일: 1열) */
    body {
      font-family: sans-serif;
      margin: 0;
      padding: 16px;
      background-color: #f8f9fa;
    }

    .card-grid {
      display: grid;
      grid-template-columns: 1fr; /* 모바일: 1열 */
      gap: 16px;
    }

    .card {
      background-color: white;
      border: 1px solid #dee2e6;
      border-radius: 8px;
      padding: 20px;
      text-align: center;
    }

    .card h3 {
      margin: 0 0 8px;
      color: #343a40;
    }

    .card p {
      margin: 0;
      color: #6c757d;
      font-size: 0.9rem;
    }

    /* 태블릿: 2열 */
    @media (min-width: 768px) {
      .card-grid {
        grid-template-columns: repeat(2, 1fr);
      }
    }

    /* 데스크톱: 3열 */
    @media (min-width: 1024px) {
      .card-grid {

```



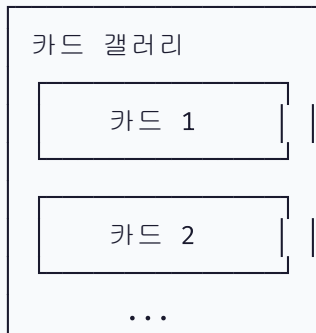
```

        grid-template-columns: repeat(3, 1fr);
    }
}
</style>
</head>
<body>
    <h1>카드 갤러리</h1>
    <div class="card-grid">
        <div class="card">
            <h3>카드 1</h3>
            <p>설명 텍스트입니다.</p>
        </div>
        <div class="card">
            <h3>카드 2</h3>
            <p>설명 텍스트입니다.</p>
        </div>
        <div class="card">
            <h3>카드 3</h3>
            <p>설명 텍스트입니다.</p>
        </div>
        <div class="card">
            <h3>카드 4</h3>
            <p>설명 텍스트입니다.</p>
        </div>
        <div class="card">
            <h3>카드 5</h3>
            <p>설명 텍스트입니다.</p>
        </div>
        <div class="card">
            <h3>카드 6</h3>
            <p>설명 텍스트입니다.</p>
        </div>
    </div>
</body>
</html>

```

브라우저 렌더링 결과

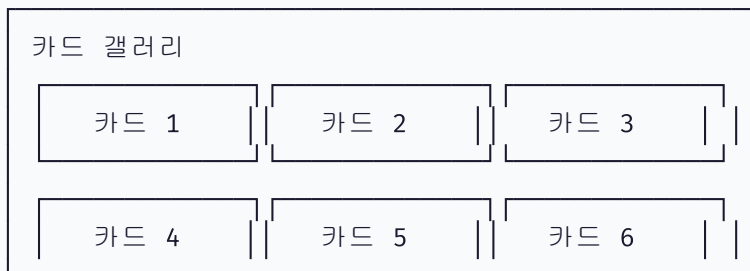
[화면 너비 < 768px - 모바일: 1열]



[화면 너비 768px ~ 1023px - 태블릿: 2열]



[화면 너비 1024px 이상 - 데스크톱: 3열]



줄별 코드 설명

줄	코드	설명
18	<code>grid-template-columns: 1fr;</code>	모바일 기본 스타일로 1열 그리드를 설정합니다
37	<code>@media (min-width: 768px) {</code>	화면 너비가 768px 이상인 경우 아래 블록을 적용합니다 (태블릿 기준)
39	<code>grid-template-columns: repeat(...</code>	너비가 동일한 2열 그리드로 변경합니다
44	<code>@media (min-width: 1024px) {</code>	화면 너비가 1024px 이상인 경우 아래 블록을 적용합니다 (데스크톱 기준)
46	<code>grid-template-columns: repeat(...</code>	너비가 동일한 3열 그리드로 변경합니다

주의: `min-width` 를 사용하면 작은 화면에서 시작하여 큰 화면으로 확장합니다(모바일 우선). `max-width` 를 사용하면 반대입니다(데스크톱 우선). 이 둘을 혼용하면 의도치 않은 충돌이 생길 수 있으므로 프로젝트 내에서 한 가지 방향으로 통일합니다.

and, not, only 조건 조합하기

미디어 쿼리는 여러 조건을 `and` 로 결합하거나 `not` 으로 부정할 수 있습니다.

```
/* 화면 미디어이고 너비가 768px 이상 1200px 이하인 경우 */
@media screen and (min-width: 768px) and (max-width: 1199px) {
  .layout {
    padding: 0 24px;
  }
}

/* 인쇄 시 불필요한 요소 숨기기 */
@media print {
  nav,
  footer {
    display: none;
  }
}
```

줄별 코드 설명

줄	코드	설명
2	<code>@media screen and (min-width: ...</code>	<code>screen</code> 미디어 타입에 너비 범위 조건 두 가지를 AND로 결합합니다
7	<code>@media print {</code>	브라우저의 인쇄 미디어에만 적용되는 스타일 블록입니다
9	<code>display: none;</code>	인쇄 시 nav와 footer를 숨깁니다

절 요약

- 미디어 쿼리(Media Query)는 `@media` 규칙으로 화면 크기 조건에 따라 CSS를 조건부 적용합니다
- 브레이크포인트(Breakpoint)는 레이아웃이 바뀌는 화면 너비 기준으로, 콘텐츠가 불편해지는 지점에서 정합니다
- `min-width` 는 모바일 우선 방향, `max-width` 는 데스크톱 우선 방향으로 사용합니다

모바일 우선 접근법

도입

미디어 쿼리를 `min-width` 로 쓰느냐 `max-width` 로 쓰느냐의 차이가 왜 중요한 것일까요? 이 선택이 바로 **모바일 우선(Mobile-first)** 전략과 **데스크톱 우선(Desktop-first)** 전략을 나눕니다. 이 절에서는 모바일 우선 접근법이 무엇인지, 왜 권장되는지, 그리고 실제로 어떻게 코드를 구성하는지 배웁니다.

핵심 개념 설명

모바일 우선이란

모바일 우선(Mobile-first)은 기본 CSS를 모바일 화면에 맞게 먼저 작성하고, 화면이 커질수록 `min-width` 미디어 쿼리로 스타일을 확장하는 설계 전략입니다. 작은 도시락에 핵심 반찬을 먼저 담고, 큰 도시락일 때 반찬을 추가하는 것과 같습니다.

데스크톱 우선은 반대입니다. 큰 화면을 기본으로 작성하고 `max-width` 로 점점 줄여 나갑니다. 직관적으로 보이지만 실제로는 불필요한 CSS를 덮어 쓰는 일이 많아 코드가 복잡해집니다.

다.

두 전략 비교

```
/* 데스크톱 우선 방식 (max-width 사용) */
.nav {
  display: flex;          /* 데스크톱 기본 */
  flex-direction: row;
}

@media (max-width: 767px) {
  .nav {
    flex-direction: column; /* 모바일에서 덮어 씁니다 */
  }
}
```

```
/* 모바일 우선 방식 (min-width 사용) */
.nav {
  display: flex;
  flex-direction: column; /* 모바일 기본 */
}

@media (min-width: 768px) {
  .nav {
    flex-direction: row; /* 태블릿 이상에서 확장합니다 */
  }
}
```

두 코드의 결과는 동일하지만 모바일 우선 방식이 더 단순합니다. 기본 스타일이 작은 화면에서 항상 올바르게 보이고, 큰 화면에서만 추가 스타일이 없이기 때문입니다.

점진적 향상 전략

모바일 우선은 **점진적 향상(Progressive Enhancement)**이라는 개념과 연결됩니다. 핵심 기능을 먼저 완벽하게 만들고, 더 큰 화면이나 더 나은 환경에서 기능을 추가해 나가는 전략입니다. 반대 개념인 **우아한 저하(Graceful Degradation)**는 최고 환경을 기준으로 만든 뒤 낮은 환경에서 어떻게 줄일지 고민합니다.

코드로 확인하기

모바일 우선 방식으로 내비게이션 바를 구현합니다. 모바일에서는 세로 메뉴, 태블릿 이상에서는 가로 메뉴로 바꿉니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>모바일 우선 내비게이션</title>
  <style>
    /* ≡ 모바일 기본 스타일 ≡ */
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: sans-serif;
    }

    .site-header {
      background-color: #1e293b;
      padding: 0 16px;
    }

    .site-header__inner {
      display: flex;
      flex-direction: column; /* 모바일: 세로 배치 */
      align-items: flex-start;
    }

    .site-logo {
      color: white;
      font-size: 1.25rem;
      font-weight: bold;
      padding: 16px 0;
      text-decoration: none;
    }

    .site-nav {
      width: 100%;
      border-top: 1px solid #334155;
    }

    .site-nav__list {
      list-style: none;
      display: flex;
      flex-direction: column; /* 모바일: 세로 메뉴 */
    }

    .site-nav__link {
      display: block;
```

```

        color: #94a3b8;
        text-decoration: none;
        padding: 12px 0;
        border-bottom: 1px solid #334155;
        transition: color 0.2s;
    }

    .site-nav__link:hover {
        color: white;
    }

    /* ≡≡≡ 태블릿 이상 (768px+) ≡≡≡ */
    @media (min-width: 768px) {
        .site-header__inner {
            flex-direction: row;          /* 가로 배치로 전환 */
            align-items: center;
            justify-content: space-between;
        }

        .site-nav {
            width: auto;
            border-top: none;
        }

        .site-nav__list {
            flex-direction: row;          /* 가로 메뉴로 전환 */
            gap: 8px;
        }

        .site-nav__link {
            padding: 20px 12px;
            border-bottom: none;
        }
    }
}
</style>
</head>
<body>
    <header class="site-header">
        <div class="site-header__inner">
            <a href="#" class="site-logo">MyBrand</a>
            <nav class="site-nav">
                <ul class="site-nav__list">
                    <li><a href="#" class="site-nav__link">홈</a></li>
                    <li><a href="#" class="site-nav__link">소개</a></li>
                    <li><a href="#" class="site-nav__link">포트폴리오</a></li>
                    <li><a href="#" class="site-nav__link">연락처</a></li>
                </ul>
            </nav>
        </div>
    </header>

    <main style="padding: 24px 16px;">

```



```
<p>화면 너비를 줄이고 늘려보면서 내비게이션 변화를 확인하세요.</p>
</main>
</body>
</html>
```

브라우저 렌더링 결과

[모바일 (< 768px)]

MyBrand
홈
소개
포트폴리오
연락처

[태블릿 / 데스크톱 (768px 이상)]

MyBrand	홈	소개	포트폴리오	연락처
---------	---	----	-------	-----

줄별 코드 설명

줄	코드	설명
10	<code>box-sizing: border-box;</code>	패딩과 테두리를 너비에 포함시켜 레이아웃 계산을 단순하게 만듭니다
24	<code>flex-direction: column;</code>	모바일 기본으로 로고와 메뉴를 세로로 쌓습니다
43	<code>flex-direction: column;</code>	모바일에서 메뉴 항목을 세로로 나열합니다
55	<code>@media (min-width: 768px) {</code>	768px 이상에서 추가 스타일을 적용합니다 (모바일 우선 방향)
57	<code>flex-direction: row;</code>	태블릿 이상에서 로고와 메뉴를 가로로 배치합니다
67	<code>flex-direction: row;</code>	태블릿 이상에서 메뉴 항목을 가로로 나열합니다

베스트 프랙티스: CSS 파일을 작성할 때 위에서 아래로 읽으면 "모바일 기본 → 태블릿 확장 → 데스크톱 확장" 순서가 되도록 구성합니다. 이 순서를 지키면 코드 흐름과 화면 크기 증가 방향이 일치하여 가독성이 높아집니다.

절 요약

- **모바일 우선(Mobile-first)**은 기본 스타일을 모바일에 맞추고, `min-width` 로 큰 화면을 확장하는 전략입니다
- 데스크톱 우선보다 코드가 단순하고 불필요한 CSS 덮어쓰기가 줄어듭니다
- **점진적 향상(Progressive Enhancement)**은 핵심 기능을 먼저 완성하고 더 좋은 환경에서 기능을 추가하는 방향입니다

유연한 단위와 레이아웃

도입

고정 픽셀(`px`) 값만 사용하면 모든 화면 크기에 완벽히 맞추기 어렵습니다. 예를 들어 데스크톱에서 `font-size: 24px` 로 지정한 제목이 작은 모바일 화면에서는 너무 크게 보일 수 있습니다. 이 절에서는 화면 크기에 따라 자동으로 조정되는 **유연한 단위(Flexible Units)**와 이를 활용한 레이아웃 기법을 배웁니다.

핵심 개념 설명

퍼센트(%)와 뷰포트 단위(vw, vh)

퍼센트(%)는 부모 요소의 크기를 기준으로 비율로 크기를 지정합니다.

뷰포트 단위(Viewport Units)는 브라우저 화면 크기를 기준으로 하는 단위입니다.

단위	기준	예시
%	부모 요소의 너비 또는 높이	<code>width: 50%</code> — 부모 너비의 절반
vw	뷰포트 너비의 1/100	<code>width: 100vw</code> — 화면 너비 전체
vh	뷰포트 높이의 1/100	<code>height: 100vh</code> — 화면 높이 전체
vmin	vw와 vh 중 작은 값	정사각형 요소에 유용
vmax	vw와 vh 중 큰 값	화면 방향에 무관한 크기 지정

주의: `100vw` 는 스크롤바 너비를 포함하므로 수평 스크롤이 생길 수 있습니다. 레이아웃 너비에는 `100%` 를 사용하고, 전체 화면 배경처럼 스크롤바가 없는 상황에서 `100vw` 를 사용합니다.

clamp()로 반응형 글자 크기 만들기

clamp(최솟값, 기준값, 최댓값) 함수는 값이 최솟값과 최댓값 사이에 있도록 자동으로 제한합니다. 미디어 쿼리 없이도 반응형 글자 크기를 만들 수 있어 매우 유용합니다.

```
h1 {
  font-size: clamp(1.5rem, 4vw, 3rem);
  /* 최소 1.5rem, 화면 너비의 4%, 최대 3rem */
}
```

화면이 작을 때는 `1.5rem`, 화면이 커질수록 뷰포트 너비의 4%로 늘다가 `3rem` 을 넘지 않습니다.

clamp(min, val, max)

항목	내용
설명	값을 최솟값과 최댓값 사이로 제한합니다. <code>max(min, min(val, max))</code> 와 동일합니다
매개변수	<code>min</code> : 허용 최솟값
	<code>val</code> : 기준값 (보통 vw 기반 유연한 값)
	<code>max</code> : 허용 최댓값
반환값	세 값 중 중간 크기의 값
사용 예시	<code>font-size: clamp(1rem, 2.5vw, 2rem)</code>

코드로 확인하기

퍼센트, `vw/vh`, `clamp()` 를 사용한 유연한 레이아웃입니다. 화면 크기를 바꾸면 글자 크기와 여백이 자동으로 조정됩니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>유연한 단위 예제</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: sans-serif;
      /* vw 기반 패딩으로 화면 크기에 따라 여백이 늘어납니다 */
      padding: 0 clamp(16px, 4vw, 64px);
    }

    /* 히어로 섹션: 화면 전체 높이를 차지합니다 */
    .hero {
      min-height: 100vh;
      display: flex;
      flex-direction: column;
      justify-content: center;
      background: linear-gradient(135deg, #667eea, #764ba2);
      color: white;
      text-align: center;
    }

    /* clamp()로 반응형 제목 크기 */
    .hero__title {
      font-size: clamp(2rem, 6vw, 5rem);
      line-height: 1.2;
      margin-bottom: clamp(12px, 2vw, 24px);
    }

    /* clamp()로 반응형 부제목 크기 */
    .hero__subtitle {
      font-size: clamp(1rem, 2.5vw, 1.5rem);
      opacity: 0.85;
    }

    /* 콘텐츠 섹션: 최대 너비 제한 */
    .content {
      max-width: 800px;          /* 지나치게 넓어지지 않도록 제한 */
      margin: 48px auto;        /* 가운데 정렬 */
      display: grid;
      gap: 24px;
    }
  
```

```

.feature {
  background-color: #f8f9fa;
  border-radius: 8px;
  padding: clamp(16px, 3vw, 32px);
}

.feature h3 {
  font-size: clamp(1rem, 2vw, 1.25rem);
  margin-bottom: 8px;
  color: #343a40;
}

.feature p {
  font-size: 0.95rem;
  color: #6c757d;
  line-height: 1.6;
}

@media (min-width: 600px) {
  .content {
    grid-template-columns: repeat(2, 1fr);
  }
}
</style>
</head>
<body>
  <section class="hero">
    <h1 class="hero__title">유연한 레이아웃</h1>
    <p class="hero__subtitle">화면 크기를 바꾸면서 글자 크기 변화를 확인하세요.</p>
  </section>

  <div class="content">
    <div class="feature">
      <h3>clamp() 함수</h3>
      <p>최솟값과 최댓값 사이에서 유연하게 크기를 조정합니다.</p>
    </div>
    <div class="feature">
      <h3>vw / vh 단위</h3>
      <p>뷰포트 크기를 기준으로 요소 크기를 지정합니다.</p>
    </div>
    <div class="feature">
      <h3>max-width</h3>
      <p>넓은 화면에서 과도하게 늘어나지 않도록 최대 너비를 제한합니다.</p>
    </div>
    <div class="feature">
      <h3>퍼센트(<!-- % -->)</h3>
      <p>부모 요소를 기준으로 비율로 크기를 지정합니다.</p>
    </div>
  </div>

```

```
</body>
</html>
```

브라우저 렌더링 결과

[모바일 (375px 기준)]



← min-height: 100vh 높이 전체

[데스크톱 (1200px 기준)]

제목 font-size ≈ 5rem (clamp 최댓값)
2열 그리드로 feature 카드 배치

줄별 코드 설명

줄	코드	설명
16	<code>padding: 0 clamp(16px, 4vw, 64px);</code>	좌우 패딩이 화면 너비의 4%(최소 16px, 최대 64px)로 자동 조정됩니다
24	<code>min-height: 100vh;</code>	뷰포트 높이 전체를 차지하는 히어로 섹션을 만듭니다
32	<code>font-size: clamp(2rem, 6vw, 5rem);</code>	작은 화면에서 2rem, 화면이 커질수록 늘어나다가 5rem에서 멈춥니다
42	<code>max-width: 800px;</code>	콘텐츠가 너무 넓게 퍼지지 않도록 최대 너비를 제한합니다
43	<code>margin: 48px auto;</code>	<code>auto</code> 좌우 마진으로 콘텐츠를 가운데 정렬합니다

min(), max() 함수

`clamp()` 와 함께 `min()` 과 `max()` 함수도 유연한 레이아웃에 활용됩니다.

```

/* 부모 너비의 80%이지만 400px을 넘지 않는 너비 */
.card {
  width: min(80%, 400px);
}

/* 최소 200px, 또는 부모의 50% 중 더 큰 값 */
.sidebar {
  width: max(200px, 50%);
}

```

팁: `width: min(100%, 600px)` 는 `max-width: 600px; width: 100%;` 와 동일한 효과입니다. 코드를 한 줄로 줄일 수 있습니다.

절 요약

- `%` 는 부모 기준, `vw/vh` 는 뷰포트 기준의 유연한 단위입니다
- `clamp(min, val, max)` 는 미디어 쿼리 없이 반응형 크기를 만드는 강력한 도구입니다

- `max-width` 로 넓은 화면에서의 지나친 확장을 방지하고, `margin: auto` 로 가운데 정렬합니다

반응형 이미지와 미디어

도입

레이아웃이 반응형이 되었다면 이제 이미지도 화면에 맞게 조정해야 합니다. 이미지가 컨테이너보다 크면 가로 스크롤이 생기고, 고해상도 이미지를 작은 모바일 화면에 그대로 내려보내면 불필요한 데이터를 낭비합니다. 이 절에서는 반응형 이미지를 다루는 방법을 배웁니다.

핵심 개념 설명

`max-width: 100%`로 이미지 넘침 방지하기

가장 단순한 반응형 이미지 기법입니다. `max-width: 100%` 를 이미지에 적용하면 컨테이너보다 커지지 않으면서도 본래 크기보다 강제로 늘어나지 않습니다.

```
img {  
  max-width: 100%;  
  height: auto; /* 비율 유지 */  
  display: block;  
}
```

베스트 프랙티스: `max-width: 100%; height: auto;` 는 모든 이미지의 기본 스타일로 전역에 적용하는 것이 좋습니다. 이미지가 실수로 컨테이너를 넘어가는 문제를 예방합니다.

`srcset`과 `sizes`로 화면별 이미지 제공하기

`max-width: 100%` 는 이미지 크기를 CSS로 조정할 뿐, 큰 이미지 파일이 전송되는 것은 막지 못합니다. 모바일 사용자에게 3000px 이미지를 내려보내면 데이터가 낭비됩니다.

srcset 속성은 브라우저에게 서로 다른 크기의 이미지 파일 목록과 각 파일의 너비를 알려줍니다. 브라우저가 화면 크기와 해상도에 맞는 최적의 이미지를 자동으로 선택합니다.

sizes 속성은 이미지가 실제로 화면에서 얼마나 넓게 표시될지를 브라우저에게 힌트로 알려줍니다.

```

```

항목	내용
설명	화면 크기와 해상도에 따라 브라우저가 최적의 이미지를 자동 선택합니다
srcset	파일경로 너비w 형식으로 이미지 후보 목록을 지정합니다 (예: small.jpg 400w, large.jpg 800w)
sizes	미디어 조건과 이미지 표시 너비를 쌍으로 지정합니다 (예: (max-width: 600px) 100vw, 50vw)
src	srcset 을 지원하지 않는 브라우저를 위한 기본 이미지 경로입니다
사용 예시	모바일에서 작은 파일, 고해상도 데스크톱에서 큰 파일을 자동 선택합니다

picture 요소로 아트 디렉션하기

srcset/sizes 는 같은 이미지의 크기를 바꿀 뿐입니다. 모바일에서는 인물 클로즈업, 데스크톱에서는 배경이 넓게 보이는 원본처럼 **이미지 자체를 바꾸고** 싶을 때는 **<picture>** 요소를 사용합니다. 이를 **아트 디렉션(Art Direction)**이라고 합니다.

```

<picture>
  <!-- 768px 이상: 가로로 넓은 이미지 -->
  <source media="(min-width: 768px)" srcset="hero-wide.jpg">
  <!-- 그 외(모바일 기본): 세로 비율 이미지 -->
  
</picture>

```

<picture> , **<source media=" ... " srcset=" ... ">**

항목	내용
설명	미디어 조건에 따라 완전히 다른 이미지를 제공하는 아트 디렉션을 구현합니다
source media	미디어 조건이 일치하면 해당 srcset 이미지를 사용합니다
source srcset	해당 조건에서 사용할 이미지 파일을 지정합니다
img (필수)	<picture> 안에 반드시 가 있어야 합니다. 어떤 source 도 맞지 않을 때의 기본 이미지이며 alt 도 여기 작성합니다
사용 예시	모바일과 데스크톱에서 완전히 다른 구도의 이미지를 보여줄 때 사용합니다

코드로 확인하기

max-width: 100% , **srcset** , **picture** 를 종합적으로 활용하는 반응형 이미지 예제입니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>반응형 이미지 예제</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: sans-serif;
      padding: 16px;
      background-color: #f8f9fa;
    }

    /* 모든 이미지에 기본 반응형 설정을 적용합니다 */
    img {
      max-width: 100%;
      height: auto;
      display: block;
    }

    h2 {
      font-size: 1rem;
      color: #495057;
      margin: 24px 0 8px;
      padding-bottom: 4px;
      border-bottom: 2px solid #dee2e6;
    }

    .demo-box {
      max-width: 600px;
      background: white;
      border-radius: 8px;
      overflow: hidden;
      margin-bottom: 24px;
      box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
    }

    .demo-caption {
      padding: 12px 16px;
      font-size: 0.85rem;
      color: #6c757d;
    }

    .img-grid {
```

```

    display: grid;
    gap: 16px;
  }

  @media (min-width: 600px) {
    .img-grid {
      grid-template-columns: repeat(2, 1fr);
    }
  }
</style>
</head>
<body>
  <h1 style="margin-bottom: 16px;">반응형 이미지</h1>

  <!-- 1. 기본: max-width: 100% -->
  <h2>기법 1: max-width: 100%</h2>
  <div class="demo-box">
    
    <p class="demo-caption">컨테이너 너비에 맞춰 자동으로 줄어듭니다.</p>
  </div>

  <!-- 2. srcset으로 해상도별 이미지 제공 -->
  <h2>기법 2: srcset / sizes</h2>
  <div class="demo-box">
    
    <p class="demo-caption">화면 크기와 해상도에 따라 브라우저가 최적 이미지를 선택
  </div>

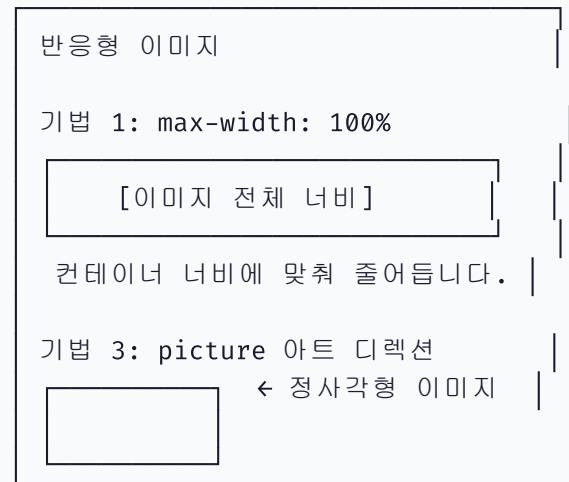
  <!-- 3. picture로 아트 디렉션 -->
  <h2>기법 3: picture 아트 디렉션</h2>
  <div class="demo-box">
    <picture>
      <!-- 768px 이상: 가로 이미지 -->
      <source
        media="(min-width: 768px)"
        srcset="https://picsum.photos/seed/wide/800/300"
      >
      <!-- 기본: 정사각형 이미지 -->
      
    </picture>
    <p class="demo-caption">768px 미만에서는 정사각형, 이상에서는 가로로 긴 이미지</p>
  </div>

<!-- 4. 이미지 그리드 -->
<h2>기법 4: 이미지 그리드</h2>
<div class="img-grid">
  <div class="demo-box" style="margin-bottom: 0;">
    
  </div>
  <div class="demo-box" style="margin-bottom: 0;">
    
  </div>
</div>
</body>
</html>

```

브라우저 렌더링 결과

[모바일 화면 (375px)]



[데스크톱 화면 (1024px+)]

기법 3에서 가로로 긴 이미지로 자동 교체됨
 기법 4에서 이미지 그리드가 2열로 배치됨

줄별 코드 설명

줄	코드	설명
17	<code>max-width: 100%;</code>	이미지가 컨테이너를 벗어나지 않도록 전역으로 설정합니다
18	<code>height: auto;</code>	너비가 줄어도 이미지 비율이 유지되도록 높이를 자동으로 조정합니다
55	<code>srcset=" ... 400w, ... 800w, ...</code>	브라우저가 선택할 수 있는 세 가지 크기의 이미지를 제공합니다
60	<code>sizes="(max-width: 600px) 100v...</code>	600px 이하 화면에서는 전체 너비, 그 이상에서는 600px로 표시됨을 브라우저에 알립니다
68	<code><source media="(min-width: 768...</code>	768px 이상 화면에서만 이 이미지를 사용하도록 지정합니다
72	<code></code>	<code><picture></code> 안의 필수 요소로, 어떤 조건도 맞지 않을 때의 기본 이미지입니다

주의: `<picture>` 요소 안의 `<img>` 태그에만 `alt` 속성을 작성합니다. `<source>` 태그에는 `alt` 를 넣지 않습니다.

팁: 실제 프로젝트에서는 여러 크기의 이미지 파일을 미리 준비해 두어야 합니다.

`400w` , `800w` , `1200w` 같은 너비 단위로 파일을 보내주면 브라우저가 적절히 선택합니다.

비교 테이블: 반응형 이미지 기법 비교

기법	방법	목적	사용 시기
<code>max-width: 100%</code>	CSS	이미지 넘침 방지	항상 기본 적용
<code>srcset</code> + <code>sizes</code>	HTML 속성	화면/해상도별 최적 파일 제공	같은 이미지의 크기 최적화
<code><picture></code>	HTML 요소	화면별로 다른 구도의 이미지 제공	아트 디렉션이 필요할 때

절 요약

- `max-width: 100%; height: auto;` 를 모든 이미지의 기본 스타일로 적용합니다
- `srcset` 과 `sizes` 로 브라우저가 화면 크기에 맞는 이미지 파일을 자동 선택하도록 합니다
- `<picture>` 요소는 화면 크기에 따라 완전히 다른 이미지를 보여주는 아트 디렉션에 사용합니다

FAQ

Q1: viewport 메타 태그 없이도 미디어 쿼리가 동작하나요?

동작하기는 하지만 의도한 대로 동작하지 않습니다. viewport 태그가 없으면 모바일 브라우저가 페이지 너비를 980px로 가정하여 렌더링합니다. 그 결과 모바일 화면에서 `min-width: 768px` 조건이 이미 참이 되어 태블릿 스타일이 적용되는 식으로 미디어 쿼리가 엉뚱하게 동작합니다. 반드시 `<meta name="viewport" content="width=device-width, initial-scale=1">` 을 포함합니다.

Q2: 브레이크포인트를 몇 개나 사용하는 것이 좋나요?

정해진 답은 없지만, 일반적으로 2~3개(모바일/태블릿/데스크톱)로 시작하는 것을 권장합니다. 브레이크포인트가 너무 많으면 유지보수가 어렵고, 코드가 복잡해집니다. 특정 기기 해상도가 아니라 콘텐츠가 보기 불편해지는 시점에서 브레이크포인트를 추가하는 방식으로 접근합니다.

Q3: `em` 단위로 미디어 쿼리를 작성하는 것이 더 좋다고 들었는데, 맞나요?

미디어 쿼리 조건에 `em` 을 사용하면 사용자가 브라우저 기본 글자 크기를 키웠을 때도 올바르게 반응한다는 장점이 있습니다. 예를 들어 `@media (min-width: 48em)` 은 기본 설정(16px)에서 768px과 같습니다. 다만 입문 단계에서는 이해하기 쉬운 `px` 단위를 먼저 사용하고, 접근성을 더 깊이 고려할 때 `em` 으로 전환해도 충분합니다.

Q4: `srcset` 을 쓰면 항상 작은 이미지를 선택하나요?

그렇지 않습니다. 브라우저는 화면 너비뿐만 아니라 장치 픽셀 비율(DPR)도 고려합니다. 레티나 디스플레이(DPR=2)의 작은 화면에서는 실제로는 큰 이미지를 선택할 수 있습니다. 또한 브라우저가 이미지를 한 번 받아 캐시했다면 더 작은 이미지가 있어도 캐시된 것을 재사용합니다.

다. `srcset` 은 브라우저에게 "이런 선택지가 있다"고 알려 주는 것이고, 최종 선택은 브라우저가 합니다.

실습 프로젝트: 반응형 카드 갤러리

프로젝트 개요

6장에서 만든 카드 갤러리를 모바일 우선 방식으로 개선합니다. 미디어 쿼리로 모바일 1열, 태블릿 2열, 데스크톱 3열 레이아웃을 구현하고, 이미지도 반응형으로 적용한 갤러리를 완성합니다.

완성 코드

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>반응형 카드 갤러리</title>
  <style>
    /* =====
    리셋 및 전역 기본 스타일
    ===== */
    *,
    *::before,
    *::after {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
      background-color: #f0f2f5;
      color: #1a1a2e;
    }

    /* 모든 이미지 반응형 기본 설정 */
    img {
      max-width: 100%;
      height: auto;
      display: block;
    }

    /* =====
    헤더
    ===== */
    .site-header {
      background-color: #16213e;
      color: white;
      padding: 0 clamp(16px, 4vw, 48px);
    }

    .site-header__inner {
      display: flex;
      flex-direction: column;
      align-items: flex-start;
      padding: 16px 0;
    }

    .site-logo {
```

```

    font-size: clamp(1.1rem, 3vw, 1.5rem);
    font-weight: 700;
    letter-spacing: -0.02em;
    margin-bottom: 12px;
}

.site-nav__list {
    list-style: none;
    display: flex;
    gap: 16px;
}

.site-nav__link {
    color: #94a3b8;
    text-decoration: none;
    font-size: 0.9rem;
    transition: color 0.2s;
}

.site-nav__link:hover {
    color: white;
}

@media (min-width: 640px) {
    .site-header__inner {
        flex-direction: row;
        align-items: center;
        justify-content: space-between;
        padding: 20px 0;
    }

    .site-logo {
        margin-bottom: 0;
    }
}

/* ===== */
히어로 섹션
/* ===== */

.hero {
    background: linear-gradient(135deg, #0f3460, #533483);
    color: white;
    text-align: center;
    padding: clamp(40px, 10vw, 80px) clamp(16px, 4vw, 48px);
}

.hero__title {
    font-size: clamp(1.75rem, 5vw, 3rem);
    font-weight: 800;
    margin-bottom: 12px;
}

```

```

.hero__subtitle {
  font-size: clamp(0.9rem, 2vw, 1.1rem);
  opacity: 0.8;
  max-width: 480px;
  margin: 0 auto;
  line-height: 1.6;
}

/* =====
   갤러리 섹션
   ===== */

.gallery {
  padding: clamp(24px, 4vw, 48px) clamp(16px, 4vw, 48px);
  max-width: 1200px;
  margin: 0 auto;
}

.gallery__heading {
  font-size: clamp(1.1rem, 2.5vw, 1.4rem);
  font-weight: 700;
  margin-bottom: 20px;
  padding-bottom: 8px;
  border-bottom: 2px solid #e2e8f0;
}

/* 카드 그리드 - 모바일: 1열 */
.card-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: clamp(12px, 2vw, 24px);
}

/* 태블릿(640px): 2열 */
@media (min-width: 640px) {
  .card-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* 데스크톱(1024px): 3열 */
@media (min-width: 1024px) {
  .card-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}

/* =====
   카드 컴포넌트
   ===== */

.card {
  background-color: white;
  border-radius: 12px;
}

```

```
    overflow: hidden;
    box-shadow: 0 1px 4px rgba(0, 0, 0, 0.08);
    transition: transform 0.2s, box-shadow 0.2s;
    position: relative;
}

.card:hover {
    transform: translateY(-4px);
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12);
}

.card__image-wrapper {
    aspect-ratio: 16 / 9; /* 이미지 영역 비율 고정 */
    overflow: hidden;
    background-color: #e2e8f0;
}

.card__image-wrapper img {
    width: 100%;
    height: 100%;
    object-fit: cover; /* 비율을 유지하며 영역을 채웁니다 */
    transition: transform 0.3s;
}

.card:hover .card__image-wrapper img {
    transform: scale(1.05);
}

.card__badge {
    position: absolute;
    top: 10px;
    right: 10px;
    background-color: #0f3460;
    color: white;
    font-size: 0.7rem;
    font-weight: 600;
    padding: 3px 8px;
    border-radius: 20px;
}

.card__body {
    padding: 16px;
}

.card__category {
    font-size: 0.75rem;
    font-weight: 600;
    color: #533483;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    margin-bottom: 6px;
}
```



```

</header>

<!-- 히어로 -->
<section class="hero">
  <h1 class="hero__title">반응형 카드 갤러리</h1>
  <p class="hero__subtitle">화면 너비를 조절해 1열, 2열, 3열로 바뀌는 레이아웃을</p>
</section>

<!-- 갤러리 -->
<main class="gallery">
  <h2 class="gallery__heading">추천 사진</h2>
  <div class="card-grid">

    <article class="card">
      <div class="card__image-wrapper">
        
      </div>
      <span class="card__badge">NEW</span>
      <div class="card__body">
        <p class="card__category">자연</p>
        <h3 class="card__title">고요한 산속 호수</h3>
        <p class="card__description">새벽 안개가 피어오르는 고요한 산속 호수의 (
      <div class="card__footer">
        <span>2026.06.01</span>
        <span>조회 1,234</span>
      </div>
    </div>
    </article>

    <article class="card">
      <div class="card__image-wrapper">
        
      </div>
      <div class="card__body">
        <p class="card__category">도시</p>
        <h3 class="card__title">빛나는 도시의 밤</h3>

```

```

        <p class="card__description">수천 개의 불빛으로 빛나는 도시의 야경을 높은
    <div class="card__footer">
        <span>2026.05.28</span>
        <span>조회 987</span>
    </div>
</div>
</article>

<article class="card">
    <div class="card__image-wrapper">
        
    </div>
    <div class="card__body">
        <p class="card__category">자연</p>
        <h3 class="card__title">숲 속 오솔길</h3>
        <p class="card__description">햇살이 나뭇잎 사이로 스며드는 조용한 오솔길
    <div class="card__footer">
        <span>2026.05.22</span>
        <span>조회 756</span>
    </div>
</div>
</article>

<article class="card">
    <div class="card__image-wrapper">
        
    </div>
    <span class="card__badge">HOT</span>
    <div class="card__body">
        <p class="card__category">자연</p>
        <h3 class="card__title">파도치는 해변</h3>
        <p class="card__description">힘차게 밀려오는 파도와 황금빛으로 물드는 일
    <div class="card__footer">
        <span>2026.05.18</span>
        <span>조회 2,310</span>
    </div>

```



```

    </div>
  </article>

  <article class="card">
    <div class="card__image-wrapper">
      
    </div>
    <div class="card__body">
      <p class="card__category">도시</p>
      <h3 class="card__title">유럽의 골목길</h3>
      <p class="card__description">역사와 현재가 공존하는 유럽 소도시의 정겨운
      <div class="card__footer">
        <span>2026.05.14</span>
        <span>조회 645</span>
      </div>
    </div>
  </article>

  <article class="card">
    <div class="card__image-wrapper">
      
    </div>
    <div class="card__body">
      <p class="card__category">자연</p>
      <h3 class="card__title">눈 덮인 설산</h3>
      <p class="card__description">순백의 눈이 덮인 웅장한 산봉우리가 하늘과 맞닿
      <div class="card__footer">
        <span>2026.05.10</span>
        <span>조회 1,089</span>
      </div>
    </div>
  </article>

</div>
</main>

```

```

<!-- 푸터 -->
<footer class="site-footer">
  <p>반응형 카드 갤러리 &copy; 2026 - ch_07 실습 프로젝트</p>
</footer>

</body>
</html>

```

줄별 코드 설명

줄	코드	설명
10	<code>*::before, *::after { box-sizi...</code>	의사 요소를 포함한 모든 요소에 border-box 설정을 적용합니다
21	<code>max-width: 100%; height: auto;</code>	모든 이미지의 반응형 기본 설정을 전역으로 적용합니다
103	<code>grid-template-columns: 1fr;</code>	모바일 기본으로 1열 그리드를 설정합니다
109	<code>@media (min-width: 640px)</code>	640px(소형 태블릿/가로 모드)부터 2열로 전환합니다
115	<code>@media (min-width: 1024px)</code>	1024px(노트북/데스크톱)부터 3열로 전환합니다
132	<code>aspect-ratio: 16 / 9;</code>	이미지 컨테이너의 가로세로 비율을 16:9로 고정합니다
138	<code>object-fit: cover;</code>	이미지가 컨테이너를 비율 유지하며 꽉 채우도록 설정합니다
163	<code>sizes="(max-width: 640px) 100v...</code>	브라우저에게 1열/2열/3열일 때의 실제 이미지 표시 너비를 알려줍니다

베스트 프랙티스: `aspect-ratio: 16 / 9` 와 `object-fit: cover` 를 함께 사용하면 이미지 크기가 달라도 카드 높이가 일정하게 유지되어 균일한 그리드를 만들 수 있습니다.

장 요약

이번 장에서는 다양한 화면 크기에서 보기 좋은 페이지를 만드는 반응형 웹 디자인의 핵심 기술을 배웠습니다. 모든 반응형 페이지의 출발점은 `viewport` 메타 태그이며, `@media` 미디어 쿼리로 화면 크기 조건에 따라 서로 다른 CSS를 적용합니다. 모바일 우선 전략으로 `min-width` 기반 브레이크포인트를 구성하면 코드가 단순해지고 유지보수가 쉬워집니다. `clamp()`, `vw/vh`, `%` 같은 유연한 단위를 활용하면 미디어 쿼리 없이도 자동으로 조정되는 요소를 만들 수 있으며, `srcset` 과 `<picture>` 요소로 이미지도 화면 크기에 최적화할 수 있습니다.

핵심 키워드

`뷰포트(Viewport)`, `미디어 쿼리(Media Query)`, `브레이크포인트(Breakpoint)`, `모바일 우선(Mobile-first)`, `clamp()`, `srcset`, `<picture>`

다음 장 미리보기

8장에서는 CSS를 더 생동감 있게 만드는 전환(transition), 변형(transform), 키프레임 애니메이션(@keyframes), 그리고 CSS 변수로 테마를 관리하는 방법을 배웁니다. 정적이었던 레이아웃에 움직임과 인터랙션을 더해 봅니다.

✂ 실습 프로젝트 (Hands-on Project)

7장 실습 프로젝트 — 반응형 카드 갤러리

이 장에서 배운 반응형 웹 디자인 기법을 실무 시나리오에 적용하는 프로젝트입니다.

초급: 반응형 카드 갤러리 (기본)

시나리오

여행 블로그를 운영 중인 친구가 사진 갤러리 페이지를 만들어 달라고 부탁했습니다. 스마트폰, 태블릿, 노트북 등 다양한 기기에서 보기 좋아야 합니다. 모바일에서는 카드가 1열로, 태블릿에서는 2열로, 데스크톱에서는 3열로 배치되어야 합니다.

학습 목표

- `viewport` 메타 태그로 반응형 기반을 설정합니다
- `@media (min-width: ...)` 미디어 쿼리로 브레이크포인트를 구현합니다
- 모바일 우선 CSS를 작성합니다
- `max-width: 100%` 로 이미지 반응형 처리를 적용합니다

진행 방법

1. `project_starter.html` 을 열고 `TODO` 주석을 찾습니다
2. 각 `TODO`를 이 장에서 배운 개념으로 완성합니다
3. 완성 후 브라우저에서 열고 창 크기를 변경하며 결과를 확인합니다

실행 방법

브라우저에서 `project_starter.html` 파일을 직접 열거나,
VS Code Live Server 확장을 사용합니다.

확인 포인트

- 브라우저 창을 768px 미만으로 좁히면 카드가 1열로 표시됩니다
- 768px~1023px 구간에서는 2열로 표시됩니다
- 1024px 이상에서는 3열로 표시됩니다
- 창 크기가 바뀌어도 이미지가 컨테이너를 벗어나지 않습니다

브레이크포인트 테스트 방법

Chrome/Edge 기준: 1. F12 키로 개발자 도구를 엽니다 2. 화면 상단의 기기 모양 아이콘(Toggle Device Toolbar, Ctrl+Shift+M)을 클릭합니다 3. 상단 드롭다운에서 "Responsive"를 선택하고 너비를 직접 입력하거나 드래그로 조절합니다

중급: 반응형 포트폴리오 갤러리 (고급)

시나리오

초급 프로젝트의 카드 갤러리를 업그레이드합니다. 클라이언트가 "사진 클릭 시 상세 오버레이가 뜨고, 카테고리 필터도 붙여 달라"고 요청했습니다. 또한 히어로 섹션에 `clamp()` 를 활용한 반응형 타이포그래피와 반응형 헤더 내비게이션을 추가합니다.

학습 목표

- `clamp()` 와 `vw` 단위로 미디어 쿼리 없이 반응형 타이포그래피를 구현합니다
- 모바일 우선 반응형 헤더 내비게이션을 구현합니다
- `srcset` 과 `sizes` 로 화면에 최적화된 이미지를 제공합니다
- `aspect-ratio` + `object-fit: cover` 로 균일한 카드 이미지를 만듭니다
- JavaScript로 카테고리 필터 기능을 구현합니다

진행 방법

1. `project_advanced_starter.html` 을 열고 `TODD` 주석을 찾습니다
2. 초급에서 연습한 내용을 바탕으로 더 복잡한 반응형 레이아웃을 완성합니다
3. 카테고리 필터 JavaScript 로직을 구현합니다
4. 카드 클릭 시 오버레이가 표시되는 기능을 완성합니다

실행 방법

브라우저에서 `project_advanced_starter.html` 파일을 직접 열거나, VS Code Live Server 확장을 사용합니다.

확인 포인트

- 헤더: 768px 미만에서 세로 내비게이션 / 768px 이상에서 가로 내비게이션
- 히어로: 창 크기에 따라 제목과 부제목 글자 크기가 자연스럽게 조정됩니다
- 카드: 640px 미만 1열 / 640px~1023px 2열 / 1024px 이상 3열
- 카테고리 필터: 버튼 클릭 시 해당 카테고리 카드만 표시됩니다
- 카드 클릭: 오버레이가 열리고 이미지·제목·설명이 표시됩니다

확장 아이디어

- 오버레이에 이전/다음 카드 이동 버튼을 추가합니다
- 헤더에 햄버거 메뉴를 구현하여 모바일 메뉴 토글을 만듭니다
- CSS `@keyframes` 로 카드 페이드인 애니메이션을 추가합니다 (8장 미리보기)
- 브레이크포인트를 더 세밀하게 나누어 4열(1280px+) 레이아웃도 추가합니다

▶  project_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_complete.html (완성 코드)

▶  project_advanced_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_complete.html (완성 코드)